

WhichCloud: Complete Project Documentation

Purpose of this document. This is the full description of the WhichCloud project, written as source material for a Software Requirements Specification (SRS) and the project report. It covers the problem, the idea, how the system works, every major module, the requirements it satisfies, how it is tested and deployed, its limitations, and how it differs from general-purpose AI assistants like ChatGPT, Claude and Gemini.

Everything here was taken from the actual source code in `github.com/itzmayank01/WhichCloud` (412 commits, Aug–Oct 2026). Facts were checked against the code. Nothing was copied from a pitch deck.

0. Quick facts

Item	Value
Project name	WhichCloud (academic title: <i>CloudForge</i>)
Full academic title	<i>CloudForge: A Constraint-Driven, LLM-Augmented Framework for Multi-Objective Cloud Architecture Synthesis and Cost Optimization</i>
Tagline	<i>Know what it costs before you build it</i>
Type	B.Tech CSE Major Project, School of Computer Science, UPES
Author / Developer	Mayank Thakur
Repository	https://github.com/itzmayank01/WhichCloud
Live demo	https://whichcloud.vercel.app
Domain	Cloud computing, FinOps (cloud financial operations), Infrastructure-as-Code, applied AI
Clouds covered	Amazon Web Services (AWS), Google Cloud Platform (GCP), Microsoft Azure
Backend	Python 3.11+, FastAPI, PostgreSQL (+ pgvector), Redis
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4, React Flow, ELK.js
Auth	Clerk (JWT verified server-side)
AI models	Google Gemini (default, free tier), Groq, Anthropic Claude, OpenAI (automatic failover chain)
Analytics	Amazon S3 + Amazon Athena + Amazon QuickSight
IaC output	Terraform (AWS, GCP and Azure)
Code size	~35,000 lines of backend Python (engine), plus the Next.js frontend
Tests	814 automated Pytest test cases collected (746 passing at the last recorded full run), plus a 13-fixture golden-total regression harness and an internal audit scorecard
Price catalog	~36,761 real price rows ingested from provider pricing feeds

1. The problem

1.1 Background

Every software team that wants to put an application on the cloud has to answer three questions before writing any infrastructure code:

1. **Which cloud?** AWS, Google Cloud or Azure.
2. **Which architecture?** Which services (virtual machines, containers, serverless, managed databases, caches, queues, CDNs, load balancers, NAT gateways...) and how they connect.
3. **What will it cost per month?** And what can be done to make it cheaper without breaking the requirements.

1.2 Why this is hard today

Pain point	What actually happens
Price calculators need an architecture first.	The AWS Pricing Calculator, Google Cloud Pricing Calculator and Azure Pricing Calculator only price what you already designed. They expect you to know you need "3 × t4g.large + db.t4g.large Multi-AZ + ALB + 500 GB egress". A student, startup founder or small team does not know that.
Each calculator covers one cloud.	Comparing AWS vs GCP vs Azure means building the same design three times in three different tools with three different vocabularies.
Thousands of SKUs.	AWS alone has 1,400+ EC2 instance types. Choosing the right one by hand is error-prone.
Hidden costs.	NAT gateway data processing, internet egress, Multi-AZ standby databases and backups are often bigger than the compute everyone looks at, and nobody forecasts them.
Optimization knowledge is scattered.	Graviton/ARM, spot instances, scale-to-zero, storage lifecycle tiering, egress-free storage (Cloudflare R2), gp3 over gp2 volumes, zram memory compression... these techniques live in blog posts, conference talks and FinOps forums. Most teams never hear of most of them.
Design and deployment drift apart.	The estimate is made in a spreadsheet; the Terraform is written later by hand with different instance sizes. The bill stops matching the estimate.
Compliance is easy to get wrong.	An Indian hospital is governed by the DPDP Act 2023 and ABDM policy, not HIPAA. A fintech regulated by RBI must keep payment data in India. Generic advice often cites the wrong law.
General AI chatbots guess prices.	Asking ChatGPT, Claude or Gemini "how much will this cost on AWS?" returns a number that the language model <i>generated from memory</i> . It may be outdated, invented, or for the wrong region, and there is no way to check it. (See Section 9.)

1.3 Problem statement

There is no single tool that takes a plain-English description of an application and its constraints, and returns several complete, compliant cloud architectures across AWS, Google Cloud and Azure — each priced line-by-line from the providers' real published rates, with measured (not claimed) savings from proven optimization techniques, an architecture diagram, and deployable Terraform that matches the estimate exactly.

1.4 Who suffers from this problem (target users)

User	Their situation
Students and fresh graduates	Learning cloud; want to know what a real architecture costs before using free credits.
Startup founders / indie developers	Need the cheapest design that still works, and need to choose a cloud.
Small and medium businesses	No dedicated cloud architect; afraid of surprise bills.
DevOps / cloud engineers	Need a fast first draft of an architecture, a cross-cloud price comparison, and starter Terraform.
FinOps / finance teams	Need to understand where the money goes and what can be saved.
Educators	Need real, checkable examples of cost trade-offs (reliability vs cost).

2. The idea (the solution)

2.1 One-line idea

Describe your app in one sentence. Get three priced cloud architectures across AWS, Google Cloud and Azure, with real line-item pricing, architecture diagrams, deployable Terraform and the optimizations that lower the bill.

2.2 Core concept: "Goal as input"

The user never fills a 40-field infrastructure form. They state a **goal and constraints in natural language**, for example:

"An online exam platform for a coaching institute in Pune, 40,000 students, busy in the evenings, results must never be lost, budget around \$400 a month."

The system:

1. **Reads** the description into structured constraints (country, sector, availability, durability, users, requests per day, peak shape, budget, storage, egress, public-facing...).
2. **Says what it guessed.** Every field it had to assume is listed with a clarifying question ("What happens to the business if this is down for an hour?").
3. **Classifies the workload** into a known shape (web app, static site, batch ETL, event-driven, ML inference, realtime/chat, VM migration) — and **refuses to price** if it cannot classify confidently, instead of guessing.
4. **Derives the load** (requests per second) from what was stated, so machines are sized from traffic, not from the budget.
5. **Filters** out any design that breaks a stated requirement *before* pricing it.
6. **Prices** every component against a catalog of real provider prices stored in PostgreSQL.

7. **Applies optimization techniques** from a curated knowledge base, prices the design with and without each one, and reports only the savings it actually measured.
8. **Returns three tiers**, each a genuinely different design (not three sizes of one design):
 - **Tier 1 – Cheapest that meets your requirements**
 - **Tier 2 – Balanced, production-ready**
 - **Tier 3 – The architecture to grow into**
9. **Draws** each architecture as a diagram (region, VPC, subnets, services) with the cost on every box.
10. **Generates Terraform** for AWS, GCP and Azure using exactly the SKUs that were priced.

2.3 The central design principle

The AI reads the request. It never sets a price.

The language model's job is limited to understanding English. Every number the user sees comes from the provider's published price list, and every saving is computed by pricing two versions of the architecture and subtracting. This is the single most important difference between WhichCloud and a general AI chatbot.

2.4 Supporting principles enforced in code

Principle	How it is enforced
Never invent a price.	If a component cannot be priced, it goes into <code>Estimate.missing</code> and the estimate is marked incomplete.
Incomplete estimates never win a comparison.	<code>compare()</code> always sorts incomplete totals last, regardless of how low they are.
Savings are measured, not claimed.	Each technique declares an <i>effect</i> (e.g. use ARM) and a <i>counterfactual</i> (x86). Both are priced; the difference is the saving. Percentages from the knowledge base are never summed.
Say what was assumed.	<code>assumed</code> fields are computed <i>after</i> extraction by subtracting what was found from what was required.
Ambiguity resolves to refusal.	An unrecognised or ambiguous workload withholds pricing and asks a question instead of producing a confident wrong bill.
A stated requirement is a filter, not a preference.	Designs that violate a stated requirement are never shown as a cheaper option.
Compliance is a lookup, not generated text.	Regulations come from a (country, sector) table, so the model cannot cite the wrong law.
Diagram and bill cannot disagree.	The diagram is built from the priced estimate, not from the request.
Terraform and estimate cannot drift.	Terraform is generated from the same resolved SKUs the estimator priced.
No hand-typed data.	Machine specs come from real catalogs; there are tests that fail if a hard-coded rate appears.

3. How it is helpful (benefits)

Benefit	Explanation
Saves days of design work	A first, complete, priced architecture in seconds instead of days of reading docs and filling calculators.
Fair multi-cloud comparison	The same requirement is priced on AWS, GCP and Azure using a service-equivalence table that refuses to compare things that are not equivalent.
Trustworthy numbers	Every line shows SKU, quantity, unit price and when the price was fetched. Users can check the arithmetic and look the row up on the Price Index page.
Teaches cost optimization	25 techniques, each with its measured saving, trade-offs and the tool that implements it (Karpenter, KEDA, Cloudflare R2, Compute Optimizer, zram...).
Makes trade-offs visible	Switching between tiers shows <i>which line changed and why</i> ("Multi-AZ database adds \$X for surviving a zone failure"), not just a bigger total.
Ready-to-use Terraform	Download a ZIP with <code>main.tf</code> , variables, modules (VPC, compute, database), outputs and a README. Built on <code>terraform-aws-modules</code> .
Compliance awareness	Flags India's DPDP Act 2023, IT Act SPDI Rules, ABDM, EHR Standards, RBI data-localisation, HIPAA (US), GDPR (EU), and locks the region where residency is required.
Honest about uncertainty	Assumptions, unpriced items and heuristic sizing are labelled everywhere, instead of being hidden.
Brownfield support	Upload a billing CSV for a waste report, scan a GitHub repo's Terraform to price it, or connect a real AWS account for live FinOps.
Free to run	Pricing works with no cloud account and no paid service; the only AI dependency defaults to Gemini's free tier, and the structured form path needs no AI at all.

4. Product features (modules from the user's point of view)

4.1 Landing page (/)

- Live showcase of priced architectures and a cross-cloud price ticker.
- Pipeline explanation: *Describe* → *Understand* → *Price* → *Optimise* → *Three options*.
- Provenance section explaining where each number comes from (exact feed, derived formula, documented multiplier).
- Pre-computed defaults embedded for instant load; data cached with incremental regeneration.

4.2 Estimate / "Price your app" (/estimate)

- **Two input paths:**
 - **Guided form** — needs no AI model, no key, returns in milliseconds and always works.
 - **Plain English** — the fast path, using an LLM to fill the same structured requirement.
- Returns three priced architectures (Cheapest / Most reliable / Most optimized) per cloud.
- Shows assumptions, clarifying questions and techniques applied/not applied with reasons.

4.3 Plan (/plan) — the constraint-driven planner

- Full pipeline: *extract* → *derive the rate* → *filter* → *price*.
- Three tiers that differ **by service**, not by size (tests require ≥ 3 services difference between consecutive tiers, or an explicit statement that nothing more is worth buying).
- **Spend priority ladder**: stated hard requirements first → sector-mandated security and audit → operational maturity → only then performance capacity.
- Compliance notes, recovery objectives (RTO/RPO by lookup), network topology decision (private subnets + NAT, public-simple, or no VPC at all).
- Terraform export of the chosen tier.

4.4 Workspace / Dashboard (/dashboard , sign-in required)

- Canvas-first layout: the architecture diagram fills the screen; a side rail shows the description, costs and reasons.
- Interactive diagram (React Flow + ELK auto-layout) with official AWS/GCP/Azure icons and a cost on every node; cost breakdown chart; tier switching that repaints the canvas.
- **Sketch mode**: drag services from a palette onto a canvas and inspect them.
- **Ask panel (Advisor)**: ask questions about the architecture on screen. The model proposes *changes* in the engine's vocabulary; the engine re-prices them. Suggestions that cannot be priced are clearly labelled "advice, no figure".
- Save, list and delete architectures (per signed-in user).
- Export diagram as standalone SVG (opens as editable shapes in draw.io / Figma).

4.5 Terraform IaC Studio (/terraform)

- Generated `main.tf`, `variables.tf`, modules (VPC, compute, database), outputs.
- Separate generators for **AWS**, **GCP** and **Azure** because their resource graphs differ in *shape* (global VPC in GCP, resource groups and delegated subnets in Azure, Cloud NAT as a single regional config, etc.).
- Reusable dev/prod environments, validate (`terraform validate`), inspect, and download ZIP.
- A component with no priced line gets no resource block; it is listed in the README under "priced but not generated" instead of being built with an invented size.

4.6 Price Index (/prices)

- The raw catalog, readable and filterable (vCPU, memory, architecture, provider, region), with the fetch date on every row. "Every other page quotes these numbers; this is where you check them."

4.7 Bill Audit (/audit)

- Upload a billing CSV / cost export (AWS, GCP, Azure formats).
- Each line is matched against knowledge-base rules; reports what each technique **would** save, priced from the catalog, with trade-off and the line it was measured against.
- Lines with no applicable technique are reported as "reviewed, nothing found", not hidden.
- Charts, filters and a cost report.

4.8 Account connections (`/connect`)

- **AWS:** cross-account IAM role with a per-connection external ID (prevents the confused-deputy problem). No access keys stored; credentials expire in one hour. Costs read via Cost Explorer. A CloudFormation/IAM role template (`whichcloud-role.yaml`) is provided.
- **GCP:** user grants WhichCloud's service account read access to their BigQuery billing export; credits are un-nested so spend is not overstated.
- **Azure:** Service Principal with Reader role; its client secret is encrypted (Fernet) and the server refuses to store one if no encryption key is configured.
- **GitHub (token scan):** reads a repo's `.tf` files, parses resources and `terraform-aws-modules` calls, and prices them. The token is used once and never stored.
- **GitHub App sign-in:** least-privilege (Contents + Metadata, read-only) installation on repositories the user picks; lists the user's own repos.

4.9 FinOps workspace (`/finops`)

- Overview with KPIs and live topology, Cost Reports, Issues (waste), Active Resources, Financial Planning, Recommendations, Settings.
- **Live for AWS** (resource scan, waste issues, planning, resource actions such as stop / terminate / release with dry-run, and a guarded "delete all" requiring a typed confirmation phrase). Destructive actions only use the caller's own connected credentials.
- For Azure, GCP and GitHub, live FinOps deliberately returns "*not implemented*" rather than showing invented figures. Some FinOps views run on clearly labelled sample data to demonstrate the workflow.

4.10 Cost analytics (AWS S3 + Athena + QuickSight)

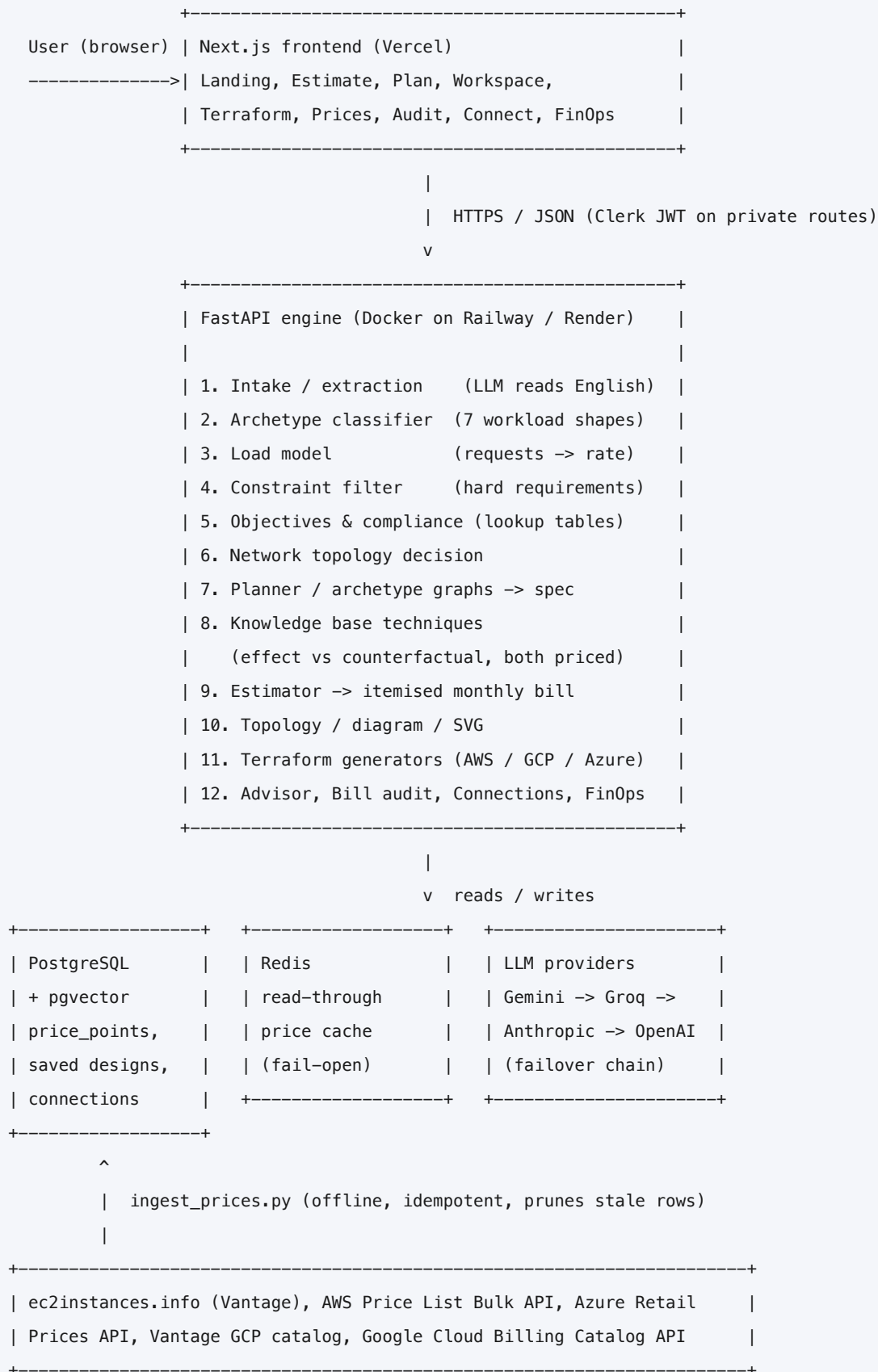
- The engine's priced output for 13 benchmark workloads (841 line items) is exported to CSV, stored in a private S3 bucket in ap-south-1 (Mumbai), queried with Athena SQL, and explored in an interactive QuickSight dashboard built **as code**.
- Findings: average monthly cost per workload rises from **\$546 (Cheapest)** → **\$657 (Balanced)** → **\$827 (Grow-into)** — reliability adds ~51%. Compute and backups are the two biggest cost drivers. 164 of 841 lines are \$0 (free tier / included). Athena tier totals match the engine's golden regression totals.

4.11 Diagram lab (`/diagram-lab` , developer tool)

- Renders each benchmark fixture's three tiers straight from the engine's topology, to prove the diagram renderer per workload and per tier.

5. System architecture

5.1 High-level flow



5.2 Layered architecture

Layer	Responsibility	Key files
Presentation	UI, diagrams, charts, forms	frontend/app/*, frontend/components/*
API	HTTP translation only, auth, CORS, rate limiting	backend/whichcloud/api.py, auth.py
Reasoning	Extraction, classification, load, filter, compliance, planning	intake.py, llm_extract.py, constraints.py, archetype.py, load_model.py, constraint_filter.py, objectives.py, network_topology.py, planner.py, plan.py, archetypes/*
Knowledge	Optimization techniques, service mappings	knowledge-base/*, knowledge.py, mappings.py
Pricing	Provider adapters, normalized price model, catalog store, cache	pricing/aws.py, azure.py, gcp.py, models.py, specs.py, store.py, cache.py
Costing	Architecture → itemised bill, cross-cloud compare	estimator.py, engine.py
Output	Topology, diagrams, SVG, Terraform	topology.py, architecture/*, terraform_export*.py
Brownfield	Bill audit, cloud connections, FinOps	billing_audit.py, connections/*, github_app.py, secrets.py
Data	PostgreSQL + pgvector, Redis	infra/docker-compose.yml

5.3 Processing pipeline in detail

1. **Extraction (Module 1 — constraints.py, llm_extract.py)** Phrase matching runs first; an LLM only fills what is left. The LLM returns *constraints only* — never a regulation name, a price, a service or a component. Results are cached by `sha256(prompt | model | schema)` so the same description always produces the same plan.
2. **Quantity audit (quantity_audit.py)** — a safety net: if the description stated a number ("40 virtual machines", "500 GB") that did not reach the constraints, pricing is refused rather than producing a wrongly-sized plan.
3. **Archetype classification (archetype.py)** — picks one of 7 implemented shapes from positive evidence only. No default/fallback; ties and no-match resolve to *unknown*, which withholds pricing. Three states: `priced`, `recognised_unpriced`, `unknown`.
4. **Load model (Module 2 — load_model.py)** — converts stated volume into a request rate and gates expensive components on it. Also records what was **not** added (e.g. "no CDN: rate too low") so omissions are visible.
5. **Network topology (network_topology.py)** — private subnets with NAT, a simple public subnet with no NAT, or no VPC at all (static site).
6. **Constraint filter (Module 3 — constraint_filter.py)** — designs that fail a stated requirement are never priced alongside compliant ones.
7. **Objectives & compliance (Module 4 — objectives.py)** — RTO/RPO and regulations by (country, sector) lookup.

8. **Planning** (`planner.py` , `plan.py` , `archetypes/*`) — each archetype declares CANDIDATES (allowed services, each with a real price in the region), SIZING driver suited to its shape, FORBIDDEN components with reasons, and three TIERS.
9. **Technique application** (`knowledge.py` , `engine.py`) — applies matching techniques; prices with and without; keeps only measured savings; lists `not_applied` with reasons.
10. **Estimation** (`estimator.py`) — itemises the bill: SKU, quantity, unit, unit price, monthly cost, fetched-at date; marks missing items.
11. **Outputs** — topology/diagram, SVG, diffs between tiers, trade-offs, Terraform.

6. Workload archetypes (what kinds of apps it can design)

Archetype	Example description	What the engine gets right
<code>web_app</code>	E-commerce site, hospital records system, coaching platform	Compute + managed DB + LB + storage + optional cache/CDN/queue/email, sized from request rate
<code>static_site</code>	"Marketing site, 30,000 visitors/month, no login, no database"	S3 + CloudFront + Route 53 + ACM; no server or database (forbidden)
<code>batch_etl</code>	"Every night we process 500 GB of sensor readings"	Billed for hours it runs (not 730/month); spot capacity; S3 → Glue → Parquet → Athena; relational DB forbidden for telemetry
<code>event_driven</code>	"40,000 payment webhooks/day in bursts, cannot drop one"	A durable queue is present even on the cheapest tier, because it is what keeps the "cannot drop one" promise
<code>ml_inference</code>	"Model scores loan applications, 50/s in business hours, none at night"	Inference endpoints; "business hours" read as traffic timing, not an uptime requirement, so it can scale down at night
<code>realtime</code>	"In-app chat for 100,000 users, history must be searchable"	Sized by peak concurrent connections (WebSockets), search index on tier 1 because search was a stated requirement
<code>migration</code>	"40 VMs, mix of Windows and Linux, move as-is"	Sized from the VM inventory; forces x86 for Windows (no Graviton); no Fargate for as-is legacy images

Each archetype was added in response to a documented "probe" prompt that the earlier engine had priced wrongly; the probe text is kept in the source as the archetype's specification.

7. Knowledge base (the "moat")

Hand-curated YAML files in `knowledge-base/techniques/` (25 techniques) and `knowledge-base/service-mappings/core.yaml` (cross-cloud equivalences).

7.1 Technique schema

```
id: kebab-case-unique-id
name: Human readable name
category: compute | storage | network | database | memory | orchestration
summary: One sentence a non-expert understands.
applies_when:           # matched against the requirement
  workload_type: [web, api, batch]
  traffic_pattern: [spiky]
  min_monthly_spend_usd: 50
savings:
  typical_pct: 30        # realistic, not best-case
  confidence: high | medium | low
  basis: Source of the number
tradeoffs:               # always listed
  - Requires ARM-compatible container images
implemented_by:          # the actual tool
  - name: Karpenter
    url: https://github.com/aws/karpenter
    type: tool | kernel-feature | managed-service
providers: [aws, gcp, azure]
obviousness: low | medium | high # low = little-known
```

Curation rules: honest savings numbers, always list trade-offs, prefer little-known techniques, every entry names a tool, cite the basis.

7.2 The 25 techniques

Technique	Category	Typical saving	Implemented by
ARM-based compute (Graviton / Ampere / Axion)	compute	~9%	docker buildx, AWS Graviton
ARM Fargate tasks	compute	~12%	docker buildx, Fargate on Graviton
Burstable instances for workloads that idle	compute	~18%	EC2 burstable instances
Commit to a year (Savings Plan / Reserved / CUD)	compute	~22%	AWS Cost Explorer
Bill scheduled compute only for hours it runs	compute	~91%	EventBridge Scheduler, AWS Batch
Size the fleet from the rate, not a round number	compute	~30%	AWS Compute Optimizer
Right-size over-provisioned instances	compute	~30%	Compute Optimizer, Goldilocks
Scale down when nobody is using it	compute	~40%	KEDA, Karpenter, Cloud Run
Spot / preemptible capacity for interruptible work	compute	~62%	Karpenter, Node Termination Handler, GCP Spot VMs

Technique	Category	Typical saving	Implemented by
ARM instances for the managed database	database	~5%	RDS Graviton classes
Cache reads instead of growing the database	database	~25%	ElastiCache
Cache reads so the database can be smaller	database	~25%	Valkey, ElastiCache
Multi-AZ database only where downtime matters	database	~50%	RDS Multi-AZ
Read replica only where reads dominate	database	~45%	RDS read replicas
ARM-based managed cache nodes	memory	~10%	ElastiCache on Graviton
zram / zswap in-memory compression	memory	~20% (advisory, unpriced)	Linux zram, zswap
Serve repeat bytes from the edge (CDN)	network	~35%	CloudFront
Serve static assets from egress-free storage	network	~90%	Cloudflare R2, rclone
Cut egress before cutting compute	network	~40%	CloudFront, co-location
One NAT gateway unless zone failure must be survived	network	~50%	VPC NAT placement
VPC endpoints instead of NAT data processing	network	~22%	AWS PrivateLink
Move cold objects to an archive class	storage	~60%	S3 Lifecycle, Intelligent-Tiering
Move cold objects to infrequent-access	storage	~20%	S3 / Azure Blob lifecycle
gp3 block storage instead of gp2	storage	~20%	EBS volume modification
Partition and compress so queries scan less	storage	~80%	AWS Glue, Apache Parquet

Important: these percentages are only shown as context. The saving reported to the user is always the **measured** difference between two priced architectures, e.g.:

✓ ARM-based compute	-\$6.57/mo	vs t3a.xlarge
✓ Spot / preemptible capacity	-\$36.21/mo	vs t4g.xlarge
measured saving	\$42.78	(32% vs untuned)

7.3 Service mappings

`core.yaml` maps equivalent services across clouds (e.g. Amazon EC2 ↔ Compute Engine ↔ Azure Virtual Machines; Fargate ↔ Cloud Run ↔ Container Instances) with a `confidence` of `close`, `partial` or `none`, plus what maps cleanly and what does not (e.g. GCP's automatic sustained-use discounts vs AWS Savings Plans). A `none` pairing is never compared; a `partial` pairing is compared with its caveat shown. This fixed an earlier bug where clouds with fewer priced components looked cheaper.

8. Pricing engine (the credibility layer)

8.1 Data sources

Source	Auth needed	Used for
ec2instances.info (Vantage, open source)	None	EC2 compute: 1,406 instance types, specs + per-region prices
AWS Price List Bulk API	None	RDS, S3, data transfer, ALB, and other AWS services
Azure Retail Prices API	None	VMs, PostgreSQL, Blob, egress, Load Balancer
Vantage GCP machine catalog	None	GCP compute: specs, on-demand, spot, committed-use rates
Google Cloud Billing Catalog API	API key	GCP storage, egress, Cloud SQL

Prices are ingested once into PostgreSQL (`price_points` table, ~36,761 rows) by `scripts/ingest_prices.py` — idempotent, and it **prunes stale rows** so a retired SKU can never be quoted. Lookups are single indexed queries; Redis sits in front as a read-through cache that fails open (if Redis is down, answers come from Postgres, just slower) and is keyed by ingest generation so a cold and warm cache give byte-identical answers.

8.2 Normalized price model

Every adapter returns a `PricePoint` (provider, service, SKU, region, unit, unit price, attributes, `fetch_at`), so the engine never learns a provider's quirks and adding a provider does not touch the engine.

8.3 Validation against independent sources

Check	Result
AWS: our catalog vs AWS Price List CSV	807 / 807 instance types match exactly (100%)
Azure: Retail Prices API vs Vantage catalog	923 / 928 match (99.5%); 5 legacy M/F-series outliers
PRD accuracy target ($\pm 20\%$ of provider calculators)	Met at 0% drift (AWS) and 0.5% (Azure)

Validation caught a real bug: an Azure SKU carries up to a dozen meters, and selecting the Windows-priced "Cloud Services" meter made 36 machine types read 2.65× too expensive. Meter selection became an allow-list; a regression test guards it.

8.4 Provenance labels shown to the user

- **Exact** — returned by the provider's pricing API and stored as it arrived.
- **Derived** — a formula over provider rates (e.g. Cloud SQL = vCPU rate × 2 + RAM rate × 8).
- **Documented multiplier** — e.g. Azure HA standby billed as a second instance (2×).

9. How WhichCloud is different from ChatGPT, Claude and Gemini

General-purpose AI assistants are excellent at explaining concepts and writing code. But for cloud architecture *costing* they have structural limitations that WhichCloud was built to remove.

9.1 Side-by-side comparison

Dimension	ChatGPT / Claude / Gemini (general chatbots)	WhichCloud
Where prices come from	Generated by the language model from training data. Can be outdated, for the wrong region, or invented ("hallucinated").	Fetches from the providers' official pricing feeds into a database. The model is never allowed to state a price.
Can you check a number?	No source, no SKU, no date.	Every line shows SKU, quantity, unit price, fetch date, and can be looked up on the Price Index page.
Freshness	Fixed at the model's training cut-off.	Re-ingested from provider APIs; stale SKUs are pruned.
Accuracy evidence	None published for pricing.	100% exact match on AWS (807 types), 99.5% on Azure, against independent second sources.
Savings claims	"Graviton saves about 20%" — a remembered percentage.	Both versions priced, difference measured, compared against a named SKU ("-\$6.57/mo vs t3a.xlarge"). Percentages never summed.
Consistency	Same question can give different architectures and totals each time.	Decision + pricing layer is fully deterministic (asserted over 100 iterations); extraction is cached so the same description gives the same plan.
Uncertainty	Tends to answer confidently even when guessing.	Lists every assumed field with a clarifying question; refuses to price unknown workload shapes.
Requirement handling	May offer a "cheaper" option that silently breaks a stated requirement.	Stated requirements are hard filters applied before pricing.
Compliance	Can cite the wrong regulation (e.g. HIPAA for an Indian hospital).	Regulations come from a (country, sector) lookup table: DPDP Act 2023, SPDI Rules, ABDM, EHR Standards, RBI localisation, HIPAA, GDPR.
Multi-cloud comparison	Rough, inconsistent, often compares non-equivalent services.	Same requirement priced on AWS, GCP and Azure using an equivalence table that refuses unfair pairings.
Diagrams	Text description, or Mermaid code you must render yourself; no costs on it.	Interactive diagram with official icons, region/VPC/subnets, and a real cost on every node; exportable SVG.
Terraform	Free-form HCL that may not run, with sizes unrelated to any estimate.	Generated from the exact priced SKUs using <code>terraform-aws-modules</code> ; validated; separate correct generators for AWS, GCP and Azure.
Hidden costs	Often omitted (NAT data processing, egress, backups, Multi-AZ standby).	Modelled explicitly, with documented ratios (e.g. NAT share of egress, CDN origin-fill share).
Your real account	Cannot see your bill or infrastructure.	Bill CSV audit, GitHub Terraform scan, and secure AWS role-based live FinOps.

Dimension	ChatGPT / Claude / Gemini (general chatbots)	WhichCloud
Role of AI	The AI is the product: it designs, prices and explains.	The AI is a narrow reader of English; a deterministic engine designs and prices. The AI can be removed entirely (form path) and the system still works.
Cost to use	Subscription for best models.	Free; pricing needs no account or key; LLM defaults to Gemini free tier with automatic failover.

9.2 In one sentence

ChatGPT, Claude and Gemini *talk about* cloud costs; WhichCloud *calculates* them — from real provider price lists, with every number traceable, every saving measured, every assumption disclosed, and Terraform that deploys exactly what was priced.

9.3 How WhichCloud uses AI responsibly (a "constrained LLM" architecture)

1. **Narrow task:** the model only converts English into a fixed schema of constraints.
2. **Schema-validated output:** structured outputs guarantee shape; Pydantic/dataclass validation on the server rejects bad values (a hallucinated field fails at the boundary).
3. **No authority over facts:** prices come from the catalog, regulations from a table, services from archetype candidate sets.
4. **Deterministic core:** everything after extraction is plain, testable Python.
5. **Advisor guardrail:** in the Ask panel, the model proposes changes; the engine re-prices them. Unpriceable suggestions are labelled as opinion.
6. **Provider-agnostic, failover chain:** Gemini → more Gemini keys → Groq → Anthropic → OpenAI. Only quota/rate-limit errors move to the next provider; keys never appear in logs.
7. **Reproducibility:** extraction cache keyed on sha256(prompt|model|schema).
8. **Measured:** intake evaluated field-by-field against hand-written fixtures (34/35 fields correct, 97%, on gemini-2.5-flash free tier); classifier accuracy and extraction variance probes are kept in `tests/probes/`.

9.4 Comparison with other existing tools

Tool	What it does	Gap WhichCloud fills
AWS / GCP / Azure Pricing Calculators	Price a design you already made, one cloud at a time	No design, no cross-cloud, no optimizations, no Terraform
Infracost	Prices existing Terraform	Needs Terraform first; breakdown deprecated, scan requires login; self-hosted pricing API moved to a paid plan
Cloud diagram generators (e.g. cloud-architect-ai, diagram-ai-generator)	Draw architectures from prompts	No real pricing, no optimizations
FinOps tools (Komiser, Cloud Custodian, OpenCost)	Find waste in existing accounts	Brownfield only; do not design new systems

Tool	What it does	Gap WhichCloud fills
ChatGPT / Claude / Gemini	Conversational advice	See 9.1

10. Functional requirements (for the SRS)

ID	Requirement
FR-1	The system shall accept a workload description in plain English.
FR-2	The system shall accept a structured requirement via a guided form without using any AI model.
FR-3	The system shall extract structured constraints: country, sector, availability, durability, users, requests/day, peak shape, budget, storage, egress, public-facing, static assets, email volume, async processing, content/user data size, and migration estate (VM count, OS mix, vCPU/RAM).
FR-4	The system shall list every assumed field and a clarifying question for each.
FR-5	The system shall classify the workload into one of the implemented archetypes, or withhold pricing if unknown/ambiguous.
FR-6	The system shall refuse to price when a stated quantity was not captured by extraction.
FR-7	The system shall derive a request rate from stated volumes and size compute from it.
FR-8	The system shall exclude any design that violates a stated requirement before pricing.
FR-9	The system shall produce three tiers that differ by service, each with a philosophy statement.
FR-10	The system shall price every component from the stored provider catalog and show SKU, quantity, unit price, monthly cost and fetch date.
FR-11	The system shall report unpriceable components as missing and mark the estimate incomplete.
FR-12	The system shall price the same requirement on AWS, GCP and Azure and compare only equivalent services.
FR-13	The system shall apply knowledge-base techniques and report measured savings against a named counterfactual SKU, and list techniques not applied with reasons.
FR-14	The system shall show differences (diffs) and trade-offs between tiers.
FR-15	The system shall show applicable regulations and recovery objectives from lookup tables, and lock the region when residency is required.
FR-16	The system shall render an architecture diagram with a cost on every node, and export it as SVG.
FR-17	The system shall generate Terraform for AWS, GCP and Azure from the priced SKUs, support validate/inspect, and download as ZIP.

ID	Requirement
FR-18	The system shall answer questions about an architecture, re-pricing any change the engine can model and labelling the rest as unpriced advice.
FR-19	The system shall let a signed-in user save, list and delete architectures; users can only access their own.
FR-20	The system shall accept a billing CSV and return waste findings with estimated monthly savings and trade-offs.
FR-21	The system shall connect AWS (IAM role + external ID), GCP (service account on BigQuery export) and Azure (encrypted service principal) accounts.
FR-22	The system shall scan a GitHub repository's Terraform and price it, without storing the token.
FR-23	The system shall provide a live AWS FinOps view (resources, issues, planning, reports) and resource actions with dry-run and typed confirmation for destructive actions.
FR-24	The system shall expose a browsable price index with filters.
FR-25	The system shall report health: catalog size, providers, last refresh, LLM key counts (counts only).

10.1 Main REST API endpoints

Method	Route	Purpose
GET	/health	Catalog size, providers, last refresh, LLM key counts
GET	/provenance	Where prices came from
GET	/regions	Supported regions
GET	/catalog	Browse prices (vCPU, memory, arch, provider filters)
GET	/techniques	Knowledge base, with priced/advisory flag
POST	/recommend	Three priced architectures for a structured requirement
POST	/compare	Same requirement priced on every cloud
POST	/describe	Plain English → three priced architectures
POST	/plan	Constraint-driven three-tier plan
POST	/plan/export.tf , /describe/export.tf	Terraform ZIP
POST	/describe/terraform/inspect , /describe/terraform/validate	Inspect / validate Terraform
POST	/architecture , /architecture/export.svg	Read/draw an architecture, export SVG

Method	Route	Purpose
POST/GET/DELETE	/architecture/save , /architecture/saved , /architecture/saved/{id}	Saved designs (auth)
POST	/advise	Ask about an architecture
POST	/audit	Billing CSV → waste report
POST	/api/connections/setup , /api/connections/verify	Cloud account connections
GET	/api/github/connect , /api/github/oauth/callback , /api/github/repos	GitHub App
POST	/api/github/webhook	GitHub App webhook
GET	/api/finops/live , /resources , /issues , /planning , /reports	FinOps (auth)
POST	/api/finops/resources/action , /api/finops/resources/delete-all	Resource actions (auth, guarded)

11. Non-functional requirements

Category	Requirement / how it is met
Accuracy	Estimates within $\pm 20\%$ of provider calculators (target); measured 0% drift AWS, 0.5% Azure. Totals labelled "estimate, not quote".
Performance	Description → three priced architectures in under 3 minutes (target); the form path returns in milliseconds; indexed Postgres lookups + Redis cache; landing page served with pre-computed defaults and edge caching.
Determinism	Same input → same architecture and price (asserted over 100 runs); stable layout ordering so diagrams never reshuffle on reload.
Reliability / availability	LLM failover chain; Redis optional (fail-open); form path works with no AI at all; graceful errors that say what to do.
Security	Clerk JWT verified against published JWKS on every per-user route (fails closed if unset); no stored AWS/GCP credentials (role/assume and service-account grants); Azure secrets Fernet-encrypted with no default key; GitHub tokens never persisted; external ID from CSPRNG to stop confused-deputy attacks; per-IP rate limiting; strict CORS allow-list; destructive actions only with caller's own credentials and typed confirmation.
Privacy	No personal data required; LLM keys never logged; warning not to paste confidential data into free-tier model requests.
Transparency / explainability	Assumptions, fetch dates, versus-SKUs, not-applied reasons, trade-offs, tier diffs, provenance labels.
Maintainability	Thin API layer; one normalized PricePoint ; knowledge base in version-controlled YAML; archetypes as declarative contracts; extensive docstrings explaining <i>why</i> .
Extensibility	New provider = new adapter only; new technique = new YAML file; new archetype = new module with candidates/sizing/forbidden/tiers.
Testability	814 Pytest cases; golden-total regression harness; internal audit scorecard; frontend layout-quality and determinism harnesses.

Category	Requirement / how it is met
Portability	Dockerised backend; docker-compose for Postgres (pgvector/pg16) + Redis; Vercel + Railway/Render deployment.
Usability	One-sentence input; responsive layout with mobile navigation; light/dark theme; reduced-motion support.
Cost of operation	Runs on free tiers; ingest-once catalog avoids live API calls on the request path.

12. Technology stack

Layer	Technologies
Frontend	Next.js 16.3, React 19.2, TypeScript 5, Tailwind CSS 4, React Flow (<code>@xyflow/react</code>), ELK.js (graph layout), Iconify, Clerk (<code>@clerk/nextjs</code>)
Backend	Python ≥ 3.11 , FastAPI, Uvicorn, Pydantic 2, httpx, psycpg 3, PyYAML, python-hcl2 (Terraform parsing), python-multipart, PyJWT, cryptography (Fernet), boto3
AI SDKs	google-genai (Gemini), anthropic (Claude), openai; Groq via compatible API
Database	PostgreSQL 16 with pgvector
Cache	Redis 7
IaC	Terraform, terraform-aws-modules (+ GCP/Azure resources)
Diagrams	React Flow + ELK in browser; server-side layered layout (Sugiyama-style) + SVG export; mingrammer/diagrams for reference PNGs
Analytics	Amazon S3, Amazon Athena, Amazon QuickSight (dashboard as code)
Testing	Pytest, golden-total regression harness, audit scorecard, tsx-based frontend harnesses
DevOps	Docker, docker-compose, Vercel (frontend), Railway / Render (backend + DB), Neon (Postgres option), Git/GitHub

13. Data design

13.1 Main entities

Entity	Key fields
PricePoint (<code>price_points</code> table)	provider, service/category, sku, region, unit, unit_price_usd, attributes (vCPU, memory, arch...), fetched_at, ingest generation
Requirement (structured input)	goal, workload_type, traffic pattern/scale, budget, latency target, region, compliance, lock-in tolerance, team skill, provider preference
Constraints (extracted)	country, country_lock, sector, availability, durability, users, requests_per_day, peak_shape, budget, storage_gb, egress_gb, public_facing, static_assets, emails_per_month, async_processing, content_storage_gb, user_data_gb, source_vm_count / OS / arch policy — each with evidence and assumed flag
ArchitectureSpec	Components with counts, sizes, AZ policy, archetype, tier

Entity	Key fields
Estimate / LineItem	label, sku, quantity, unit, unit_price, monthly_cost, fetched_at; total; missing list; complete flag
Tier	name, label, philosophy, spec, estimate, fingerprint, diffs, trade-offs
Technique	id, name, category, applies_when, savings, tradeoffs, implemented_by, providers, obviousness, effect, counterfactual
Topology	nodes (kind, service, cost, tier, zone) and edges (flow type)
SavedArchitecture	id, owner (from verified JWT), description, payload, created_at
Connection	owner, provider, role ARN / project / subscription, external id, encrypted secret (Azure only), status
GitHubInstallation	owner ↔ installation id mapping

13.2 Neutral regions

Countries map to neutral region keys (e.g. IN → `india`, `india-south`; SG → `singapore`; US → `us-east`; IE/DE/FR/GB → `eu-west`), which each adapter translates to its own region (AWS `ap-south-1`, Azure `centralindia`, GCP `asia-south1`, ...).

14. Use cases

ID	Actor	Use case	Outcome
UC-1	Visitor	Describe an app in one sentence	Three priced tiers per cloud, diagram, assumptions
UC-2	Visitor	Fill the guided form	Same result, no AI involved
UC-3	Visitor	Compare clouds	Cheapest equivalent design per cloud and the difference
UC-4	Visitor	Switch tiers	See exactly which lines changed and what reliability each buys
UC-5	Visitor	Download Terraform	ZIP with modules matching the estimate
UC-6	Visitor	Browse the price index	Verify any number on the site
UC-7	Signed-in user	Save / reopen / delete an architecture	Personal workspace
UC-8	Signed-in user	Ask "should I use spot here?"	Re-priced suggestion or labelled advice
UC-9	User	Upload a billing CSV	Waste report with savings and trade-offs
UC-10	User	Connect AWS account	Live resources, issues, planning, safe actions
UC-11	User	Connect GitHub repo	Terraform in the repo priced line by line
UC-12	Admin/Dev	Ingest prices	Catalog refreshed, stale rows pruned

ID	Actor	Use case	Outcome
UC-13	Admin/Dev	Run regression harness	Golden totals and fingerprint checks pass/fail

Example end-to-end scenario

Input: "An e-commerce site for 50k users, spiky weekend traffic, \$400/mo." Output (AWS, ap-south-1, Balanced, verified run):

Compute × 3	t4g.large	\$98.11	2190 × \$0.0448/hour
Database	db.t4g.large	\$121.91	730 × \$0.1670/hour
Object storage	s3:general-purpose	\$5.00	200 × \$0.0250/GB-month
Egress	egress:internet	\$54.65	500 × \$0.1093/GB
Load balancer	alb	\$17.45	730 × \$0.0239/hour
Total		\$297.12	
Azure centralindia		\$338.14	→ AWS cheaper by \$41.02/mo (12%)

15. Testing and quality assurance

Layer	What it checks
Unit / integration tests (Pytest, 814 cases)	Pricing adapters (AWS, Azure, GCP), price cache equivalence, mappings, engine, determinism, budget invariance (no tier dearer at a lower budget), compliance lookup, archetypes and their graphs, serverless, event-driven, migration correctness, Terraform for AWS/GCP/Azure, Terraform matches estimate , topology, diagram layout, API, rate limiting + CORS, LLM reader failover, intake providers, extraction units, quantity audit, no hard-coded rates, disclosure, GitHub connections and GitHub App. No test calls a paid model API.
Golden-total regression harness (tests/run_harness.py)	13 benchmark workloads (e.g. hospital in Pune, fintech in Bengaluru, coaching platform, e-commerce scale, internal low-stakes tool, static site, batch ETL, webhooks, ML inference, realtime chat, VM migration, budget-floor conflict, catalog region integrity) checked against recorded totals and hundreds of assertions each.
Architecture fingerprints	<i>Divergence</i> (different workloads → different designs, proving outputs are derived not templated), <i>tier spread</i> (≥3 services between tiers), <i>stability</i> (same input → same fingerprint).
Audit scorecard (backend/audit)	Scores architecture correctness and cost accuracy separately: template triage, sensitivity to one added clause, distance from vendor reference architectures, rate→total arithmetic, consistency checks, cost-driver analysis, budget inversion.
Pricing validation scripts	Cross-source price checks (AWS 100%, Azure 99.5%).
Intake evaluation	Field-by-field scoring of LLM extraction (97% on fixtures); classifier accuracy and extraction variance probes.
Frontend harnesses	Diagram layout quality, determinism, cost report checks.

16. Deployment

Part	Host
Next.js frontend	Vercel (auto-deploy on push to <code>main</code>)
FastAPI engine	Docker on Railway or Render (Blueprint in <code>render.yaml</code>)
PostgreSQL + pgvector	Railway / Neon
Redis	Railway (optional)
Auth	Clerk (production instance for the live domain)

Key environment variables: `WHICHCLOUD_DSN` , `WHICHCLOUD_REDIS_URL` , `GEMINI_API_KEY` (+ `_2.._9` , `GROQ_API_KEY` , `ANTHROPIC_API_KEY` , `OPENAI_API_KEY`), `CLERK_JWKS_URL` , `WHICHCLOUD_ALLOWED_ORIGINS` , `NEXT_PUBLIC_API_URL` , Clerk keys, and the deployment's own AWS identity for AssumeRole. Price catalog is ingested once from a developer machine against the deployed database.

Local run:

```
cd backend
python3 -m venv .venv && .venv/bin/pip install -e .
docker compose -f ../infra/docker-compose.yml up -d
.venv/bin/python scripts/ingest_prices.py --region india
.venv/bin/python scripts/recommend.py --goal "e-commerce site" --scale medium --spiky --budget 400
.venv/bin/python -m pytest -q
```

17. Constraints, assumptions and scope

17.1 Design constraints

- Only publicly available list / spot prices; no negotiated enterprise discounts (EDPs).
- Estimates are directional and labelled "estimate, not quote".
- The system generates IaC but **never runs terraform apply** on the user's behalf.
- Sizing rules (`BASE_SIZING` , storage/egress defaults per sector) are engineering heuristics, clearly labelled as such and collected in one place so they can be tuned.

17.2 Assumptions

- Users can describe their workload in English (or use the form).
- Provider pricing feeds remain publicly accessible.
- At least one LLM key is configured for the plain-English path.

17.3 Out of scope (current version)

- Automatic provisioning / deployment.
- Continuous monitoring, drift detection and alerts.
- Billing/payments for the product itself.

- Every niche cloud SKU (core categories only).
- Multi-cloud orchestration (WhichCloud compares clouds; it does not manage them).
- Team/org features (roles, sharing, RBAC).

18. Known limitations (honest list)

Limitation	Impact
GCP storage, egress and Cloud SQL pricing need a Google API key; GCP has no second credential-free source for validation	GCP numbers are labelled unvalidated / incomplete when the key is absent
AWS spot price feed has no timestamp	Fine for ranking spot vs on-demand, not billing-grade
Azure HA is derived (standby = second instance, 2x)	Labelled as a documented multiplier
List prices only by default	Commitments shown as a separate technique
Live FinOps is AWS-only	Azure/GCP live views return "not implemented" instead of fake data; some FinOps screens use labelled sample data
Most detailed architecture diagrams and Terraform are richest for AWS	GCP/Azure generators exist but cover curated common patterns
Free LLM tiers are small (e.g. 20 Gemini requests/day/key)	Mitigated by key chain and extraction cache; form path needs no LLM
Sizing is heuristic	Labelled everywhere; replaceable by measured load later

19. Future scope

1. More archetypes and composite workloads (e.g. web app + ML + batch together).
2. Full GCP and Azure parity for diagrams, Terraform and live FinOps.
3. Committed-use / Savings Plan pricing modelled side-by-side with on-demand.
4. Continuous price refresh with change alerts.
5. Live cost tracking vs estimate after deployment (closing the loop).
6. Carbon footprint estimation per architecture.
7. More regions and countries in the compliance table.
8. Team workspaces and sharing.
9. Kubernetes (EKS/GKE/AKS) archetypes with Karpenter/KEDA-aware sizing.
10. RAG over the knowledge base with pgvector for the Advisor.

20. Development history (milestones from git log)

Period	Milestone
Aug 2026	PRD v1; pricing engine built and validated without Infracost; AWS + Azure adapters; knowledge base; plain-English intake (Gemini/Claude); FastAPI API

Period	Milestone
Aug–Sep 2026	Constraint-driven planner (4 reasoning modules); archetypes for six new workload shapes; compliance/objectives tables; GCP compute pricing; Redis cache; Terraform for AWS, GCP, Azure; diagrams with React Flow + ELK; SVG export; Clerk auth; tenant-isolation fix
Sep 2026	Cloud connections (AWS role, GCP, Azure, GitHub); live AWS FinOps; bill audit; cost analytics on S3/Athena/QuickSight; regression harness and audit scorecard; deployment on Vercel + Railway/Render
Sep–Oct 2026	GitHub App sign-in and repo listing; Terraform scanning of repos; landing performance (instant load with pre-computed defaults); reusable dev/prod Terraform environments

21. Glossary

Term	Meaning
Archetype	A workload shape the engine knows how to build (web app, static site, batch ETL...)
Counterfactual	The un-optimized alternative a technique is priced against
CDN	Content Delivery Network (e.g. CloudFront)
Egress	Data leaving a cloud network, billed per GB
FinOps	Cloud financial operations — managing and optimizing cloud spend
Graviton / Ampere / Axion	ARM-based processors on AWS / Azure / GCP
IaC	Infrastructure as Code (Terraform)
Multi-AZ	Running a standby in a second availability zone for high availability
NAT gateway	Lets private subnets reach the internet; billed per hour and per GB
PricePoint	WhichCloud's normalized price record
RTO / RPO	Recovery Time Objective / Recovery Point Objective
SKU	Stock-keeping unit — a specific billable product (e.g. t4g.large)
Spot / preemptible	Discounted spare capacity that can be reclaimed
Tier	One of the three generated designs (Cheapest / Balanced / Grow-into)
DPDP Act	India's Digital Personal Data Protection Act, 2023

22. Prompt to give ChatGPT along with this document

*"Using the project documentation below, write a complete Software Requirements Specification (SRS) for WhichCloud following IEEE 830 / ISO/IEC/IEEE 29148 structure:

1. Introduction (purpose, scope, definitions, references, overview);
2. Overall description (product perspective, product functions, user classes, operating environment, design and implementation constraints, assumptions and dependencies);
3. Specific requirements (external interfaces — user, hardware, software, communication; functional requirements with IDs; non-functional requirements — performance, security, reliability, availability, maintainability, portability, usability);
4. Use case descriptions and diagrams (describe them in

- text/PlantUML); 5. Data flow diagrams (level 0 and level 1) and ER diagram in text/Mermaid;
- 6. Comparison with existing systems including ChatGPT, Claude and Gemini;
- 2. Appendices (glossary, limitations, future scope). Use only facts from the document. Do not invent features, numbers or results. "*"